

AI-Native Grocery Merchant

Document 01 — Development Master Specification

Status	LOCKED — Development Source of Truth
Purpose	Engineering implementation and scope control
Product	AI-Native Grocery Merchant
Track	Razorpay Buildathon — Track 01: AI Growth & Agentic Commerce
Payment	Razorpay Test Mode
Audience	Developer / AI coding assistant / technical reviewer

Core principle: The AI reasons. The user chooses. The policy engine authorizes. Razorpay executes. The audit trail records.

1. Purpose

This document is the master engineering specification for the locked MVP. It defines what must be built, what must not be built, the authoritative system boundaries, the canonical end-to-end behavior, and the definition of done. It is intended to prevent scope drift during implementation.

This is a development document, not a pitch deck, judging guide, investor document, marketing plan, or production-scale architecture specification.

2. Product Definition

AI-Native Grocery Merchant is a bounded AI commerce system in which a user gives an AI buyer a grocery goal and an authorized budget. The AI determines required items and quantities, researches available products and quality signals, optimizes pack combinations and cost, evaluates merchant vouchers and loyalty benefits, and produces two optimized purchase options: Best Value and Best Quality Within Budget. The AI recommends one option, but the user makes the final basket choice. The selected basket is independently validated by a deterministic policy engine before a Razorpay Test Mode payment is initiated.

Product statement: Give AI your grocery goal, not a shopping list. It researches, optimizes and shops for you — while never exceeding the authority you gave it.

3. Locked MVP Scope

Capability	Requirement
Natural-language goal	User states a grocery objective, servings/quantity context, quality preference if supplied, and budget.
Requirement extraction	AI converts the goal into structured requirements and quantities.
Catalog	Controlled grocery catalog with authoritative prices, stock, pack sizes and metadata.
Product research	AI evaluates available quality/review evidence and distinguishes evidence from conclusions.
Quantity/pack optimization	Combine pack sizes to satisfy requirements without unnecessary purchases.
Deal optimization	Evaluate price, quantity, quality and merchant benefits.
Smart voucher decision	Choose USE or SAVE based on meaningful current value and known constraints.
Loyalty decision	Evaluate eligible rewards and whether consuming them is sensible.
Best Value	Lowest practical cost while maintaining acceptable quality.
Best Quality	Highest practical quality/reliability within authorized budget.
AI recommendation	Recommend one basket with a concise explanation.
User choice	User selects the basket before payment.
Mandate	Server-side limits for category, spend, item limits, purpose and validity.
Policy engine	Deterministic authorization of the selected final basket.

Capability	Requirement
Cart	Server-authoritative basket and totals.
Razorpay	Test Mode order creation, checkout, payment verification and webhooks.
Audit	Record meaningful AI proposals, policy decisions, incentive decisions and payment state.
Graceful failure	At least one policy denial is demonstrated and recovered from.

4. Explicitly Out of Scope

- Multi-merchant marketplace or multi-agent architecture.
- Voice commerce, negotiation engine, autonomous merchant negotiation.
- Production payments, real-money transactions, autonomous refunds, payouts or banking operations.
- Full ACP/AP2/UAP implementation or certification; protocol-inspired architecture is acceptable, unsupported compliance claims are not.
- General-purpose web-shopping crawler or unrestricted web agent.
- Large catalog, large-scale recommendation infrastructure or thousands of concurrent sessions.
- Full loyalty platform or generic external coupon marketplace.
- Complex analytics dashboard, CRM/ERP integration or merchant back-office suite.

5. Canonical User Journey

```

USER GOAL + BUDGET
↓
REQUIREMENT EXTRACTION
↓
CATALOG SEARCH
↓
PRODUCT / QUALITY RESEARCH
↓
QUANTITY + PACK OPTIMIZATION
↓
DEAL + VOUCHER + LOYALTY OPTIMIZATION
↓
BEST VALUE + BEST QUALITY
↓
AI RECOMMENDATION
↓
USER CHOOSES
↓
SERVER-SIDE REVALIDATION
↓
DETERMINISTIC POLICY ENGINE
↓
ALLOW / DENY
↓
RAZORPAY TEST MODE
↓
PAYMENT VERIFICATION / WEBHOOK
↓
AUDIT + ORDER CONFIRMATION

```

6. Basket Optimization

Best Value: Find the lowest-cost combination that satisfies required quantities and constraints without falling below the acceptable quality/reliability threshold.

Best Quality: Find the highest practical quality/reliability combination that satisfies requirements and remains within the user's authorized budget.

Budget is a hard ceiling, not a spending target. The AI must not add unnecessary products merely because budget remains.

Quality claims must be evidence-based. The implementation should distinguish merchant/catalog facts, review/evidence signals, and the AI conclusion. The MVP should use controlled or seeded evidence rather than becoming a general-purpose crawler.

7. Smart Voucher & Loyalty Rules

- Voucher availability does not imply voucher usage.
- Calculate the actual current saving before recommending USE.
- Consider eligibility, minimum-spend rules, expiry and known future value.
- A small discount may be better saved. Example: a 5% voucher saving █5 on a █100 snack may be saved.
- Never add unnecessary products solely to unlock a voucher or reward.
- Loyalty rewards follow the same value-preservation principle.
- Final payable amount is server-calculated after incentives.
- Record gross basket value, discount value and final payable separately.

8. Authority & Trust Model

The LLM is intelligent but untrusted. It can propose actions and explanations, but it cannot authorize financial actions.

Component	Can do	Cannot be authoritative for
AI Agent	Requirements, candidates, optimization, recommendation, explanations	Mandates, price, stock, final amount, payment authorization
Frontend	Display, interaction, basket selection request	Authorization, final price, payment amount
Commerce Core	Catalog, cart and order state	AI subjective reasoning
Policy Engine	ALLOW / DENY financial action	LLM reasoning or inferred preference
Razorpay	Payment execution/status	Product selection or mandate semantics
Audit	Record key events/decisions	Authorizing a transaction

9. Mandate Model

A mandate is the user's explicit financial authority granted to the agent. It is stored and enforced server-side. The AI cannot modify it.

```
{
  "agent_id": "agent-001",
  "mandate_id": "mandate-001",
  "max_spend": 1000,
  "allowed_categories": ["grocery"],
  "max_per_item": 300,
  "valid_until": "2026-09-07T23:59:59Z",
  "purpose": "pasta ingredients for 4"
}
```

10. Policy Engine Contract

The policy engine independently validates the selected final basket. It must run before any money action and again immediately before checkout/order creation.

```
MANDATE_VALID
↓
CATEGORY_ALLOWED
↓
STOCK_AVAILABLE
↓
MAX_PER_ITEM
↓
INCENTIVE / VOUCHER VALIDITY
↓
FINAL_AMOUNT_CALCULATION
↓
MAX_SPEND
↓
```

```
ALLOW / DENY
↓
AUDIT DECISION
↓
CART / CHECKOUT ONLY IF ALLOWED
```

Spend rule: Maximum spend applies to the final amount charged to the user. Gross basket value and discounts remain separately recorded.

11. Canonical Example

User: “I want to make pasta for 4 people. My budget is ₹1,000. Get good-quality ingredients and try to get me the best deal.”

Stage	Expected behavior
Intent	AI identifies pasta meal for four and ₹1,000 hard budget.
Requirements	AI derives ingredient categories and approximate quantities.
Research	AI compares candidates using price, pack size and quality/review evidence.
Optimization	AI can reject a cheaper poor-quality option and select a better combination.
Incentives	AI evaluates voucher and loyalty value instead of blindly applying them.
Two baskets	Best Value minimizes practical cost; Best Quality maximizes practical quality within ₹1,000.
Recommendation	AI recommends one basket and explains why.
User	User chooses the preferred basket.
Authorization	Policy engine validates the selected final basket.
Payment	Backend creates Razorpay Test Mode order using server-calculated amount.
Verification	Payment is verified and webhook state is reconciled.
Audit	Important decisions and financial state changes are recorded.

12. Quality / Pack Optimization Example

For a requirement of six eggs:

Option	Quantity	Cost	Quality signal	Decision
A	6 × ₹6	₹36	Low rating / quality concerns	Reject
B	6 × ₹7	₹42	Good quality	Accept
C	3 × 2-pack × ₹12	₹36	Good quality	Preferred

Expected reasoning: A is nominally cheap but fails the acceptable-quality threshold. C satisfies the same quantity at the same ₹36 cost while carrying a better quality signal, demonstrating value optimization rather than blind cheapest-SKU selection.

13. Failure Handling

At least one failure must demonstrate that the AI cannot bypass the policy engine. The canonical failure is a budget violation.

```
Mandate: Maximum spend = ₹800
AI proposal: Final basket = ₹850
Policy Engine: DENY - MAX_SPEND - Final payable ₹850 exceeds authorized maximum ₹800
AI recovery: Re-optimize and produce a compliant alternative, for example ₹774, without changing the
mandate.
```

The denial must occur server-side. A client-side warning is not sufficient.

14. Payment & Financial Integrity

- Only the backend may create the Razorpay order.
- The amount sent to Razorpay must be calculated from authoritative server-side basket state.
- The browser and LLM must not supply the authoritative final amount.
- Final checkout must revalidate price, stock, incentives and mandate.

- Payment success must be verified server-side and reconciled with webhook state.
- Webhook processing must be idempotent.
- Secrets and API keys must never be committed to source control.
- Use Razorpay Test Mode; no real money is involved.

15. Audit Requirements

The audit trail is part of the trust boundary, not merely debug logging. It should record meaningful financial and authorization events.

```
{
  "transaction_id": "txn-001",
  "agent_id": "agent-001",
  "mandate_id": "mandate-001",
  "action": "basket_selection",
  "decision": "ALLOW",
  "gross_total": 920,
  "discount_total": 46,
  "final_payable": 874,
  "voucher_decision": "USE",
  "policy_result": "ALLOW"
}
```

16. Development Principles

- **Backend authoritative:** price, stock, incentives, totals, mandate and authorization live on the server.
- **LLM untrusted:** use structured tool calls and validated outputs for financial workflows.
- **Deterministic money path:** no LLM-generated amount directly reaches Razorpay.
- **Evidence over hallucination:** quality claims map to available evidence.
- **Recoverable failures:** policy denials return structured reasons the AI can use to re-plan.
- **Explainable decisions:** important optimization and authorization decisions have concise reasons.
- **Test critical paths:** policy, totals, incentives, payment state and webhooks require automated tests.
- **Scope control:** implement the locked MVP before adding anything new.

17. Development Boundaries

Layer	Responsibility
Frontend	Goal, mandate, basket comparison, recommendation explanation, selection, payment UI and order state.
AI Agent	Intent extraction, catalog queries, research synthesis, optimization, incentive reasoning, proposals and recovery.
Commerce Core	Catalog, product facts, cart, authoritative totals and order state.
Optimization Services	Deterministic calculations for quantities, pack combinations, discount math and scoring where practical.
Policy Engine	Mandate and transaction authorization; structured ALLOW/DENY.
Payment Adapter	Razorpay Test Mode order creation, verification and webhook mapping.
Audit	Append-oriented record of key decisions and financial state changes.

18. Definition of Done

- User can provide a natural-language grocery goal and budget.
- System extracts requirements and searches the controlled catalog.
- AI demonstrates at least one meaningful quality-vs-price decision.
- AI demonstrates quantity/pack optimization.
- AI evaluates a voucher and chooses USE or SAVE with a reason.
- AI evaluates an available loyalty reward with a reason.

- System generates Best Value and Best Quality baskets.
- AI recommends one basket and explains the recommendation.
- User can select the basket.
- Final basket is recalculated server-side.
- Policy engine independently validates the transaction.
- At least one policy denial is demonstrated and AI recovers with a compliant alternative.
- Razorpay Test Mode order creation works using the server-calculated amount.
- Payment verification/webhook handling works.
- Audit records key decision and payment events.
- Critical policy, incentive, cart-total and payment-state paths have automated tests.

19. Development Order

```

PHASE 0 – Repository + environment
PHASE 1 – Catalog + authoritative product data
PHASE 2 – Mandate + policy engine + audit
PHASE 3 – Cart + server-side totals
PHASE 4 – Razorpay Test Mode + verification/webhooks
PHASE 5 – AI intent extraction + catalog tools
PHASE 6 – Product research + quantity/pack/deal optimization
PHASE 7 – Voucher + loyalty decision engine
PHASE 8 – Best Value + Best Quality + recommendation + user selection
PHASE 9 – Policy revalidation + failure recovery
PHASE 10 – End-to-end testing + demo hardening

```

20. Scope Lock

This specification represents the final locked MVP scope. New ideas discovered during implementation must not be added automatically. Any proposed change must first be evaluated against scope, engineering cost, reliability and the core Track 01 objective. The default decision is to preserve the locked scope.

Locked architecture: AI generates and optimizes → AI recommends → user chooses → policy engine authorizes → Razorpay pays → audit records.

21. Related Development Documents

- 02 — Functional Requirements Specification
- 03 — System Architecture & Technical Design
- 04 — AI Agent Specification
- 05 — Optimization & Decision Specification
- 06 — Policy & Mandate Specification
- 07 — Data Model & Database Specification
- 08 — API Contract Specification
- 09 — Razorpay Integration Specification
- 10 — Testing & Acceptance Specification
- DEVELOPMENT_ROADMAP.md
- DECISION_LOG.md